Dockerized Sports AI Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Submitted by:

Sr no	Student Name	Enrolment Number
1.	AANCHAL MARATHE	EN22CS301015
2.	AAHMAD BURHANI	EN22CS301010
3.	ABHISHEK SANKHALA	EN22CS301035
4.	YASHASVI MODI	EN22CS3011114
5.	AKSHAT YADAV	EN22CS301095

Group Name:10 D1

Group Number: DO-10

Industry Mentor Name:

University Mentor Name: Dr. Ritesh Joshi

Academic Year: 2026

Table of Contents

1.	**Problem Statement & Objectives**	**3**
	1.1 Problem Statement	
	1.2 Project Objectives	
	1.3 Scope of the project	
2.	**Proposed Solution**	**4**
	2.1 Key features	
	2.2 Overall Architecture / Workflow	
	2.3 Tools & Technologies Used	
3.	**Results & Output**	**6**
	3.1 Screenshots / outputs	
	3.2 Reports / dashboards / models	
	3.3 Key outcomes	
4.	**Conclusion**	**10**
5.	**Future Scope & Enhancements**	**11**

1. Problem Statement & Objectives

1.1 Problem Statement

Modern AI-based applications, especially those built using frameworks like CrewAI and LangChain, often face challenges in deployment and scalability. These applications depend on multiple components such as APIs (e.g., TheSportsDB), environment configurations, and external services like OpenAI models.

When deployed in traditional environments, such systems suffer from:

- Inconsistent runtime environments across different machines
- Dependency conflicts and version mismatches
- Difficulty in scaling and maintaining the application
- Complex deployment procedures for team collaboration

Additionally, AI-driven applications with agent-based architectures require efficient resource management and orchestration, which is difficult to achieve without containerization and cloud-based solutions.

To address these challenges, there is a need to design a system that ensures **portability, scalability, and consistency** across development and production environments using modern DevOps practices such as containerization and orchestration.

1.2 Project Objectives

The primary objectives of this project are:

- To develop an AI-powered sports service application using Python and agent-based frameworks
- To containerize the application using Docker for consistent deployment
- To create a lightweight and optimized Docker image using best practices
- To deploy the application on cloud infrastructure using Amazon Web Services
- To orchestrate containers using Kubernetes for scalability and high availability
- To ensure secure handling of API keys and environment variables
- To implement a structured and automated deployment workflow
- To enable easy collaboration among team members through standardized environments

1.3 Scope of the Project

The scope of this project includes the design, development, and deployment of a containerized AI-based sports service system. The project covers:

- Development of a web-based interface using Streamlit
- Integration of AI agents using CrewAI and LangChain
- Use of external APIs for real-time sports data retrieval
- Containerization of the application using Docker
- Deployment on a cloud-based virtual server (AWS EC2)
- Implementation of Kubernetes for container orchestration
- Basic CI/CD pipeline setup for automated build and deployment

However, the project is limited to:

- A prototype-level implementation and not a full-scale commercial product
- Basic security configurations without advanced enterprise-level security mechanisms
- Limited scalability testing due to resource constraints

Despite these limitations, the project successfully demonstrates the practical implementation of modern DevOps practices in deploying AI-powered applications.

2. Proposed Solution

2.1 Key Features

The proposed solution is an AI-powered sports service application that integrates modern DevOps practices to ensure scalability, portability, and reliability. The system combines agent-based AI with containerized deployment and cloud infrastructure.

Key features of the system include:

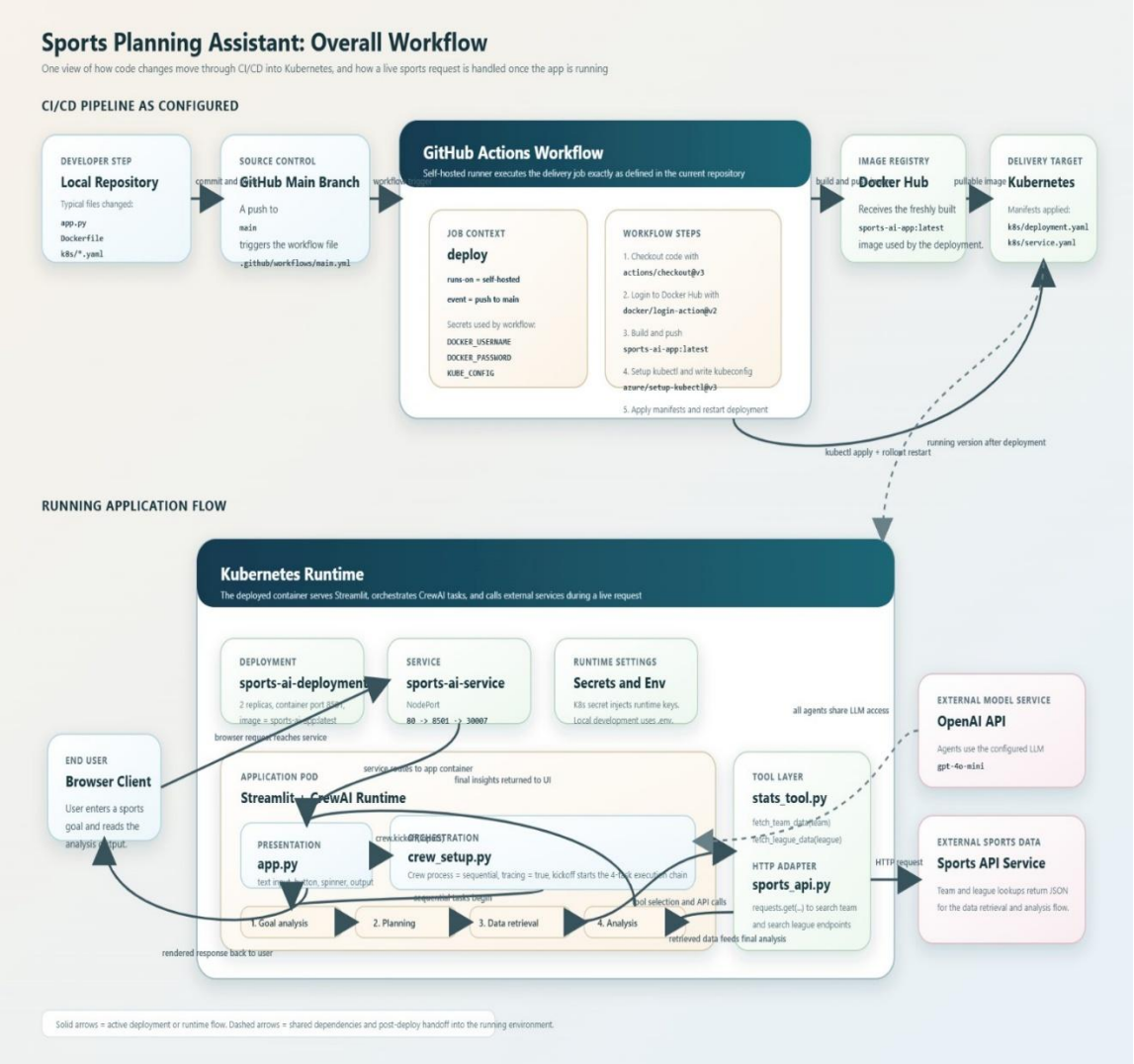
- **AI-driven Sports Insights:**
  Uses agent-based frameworks like CrewAI and LangChain to analyze and process user queries intelligently.
- **Interactive User Interface:**
  Built using Streamlit to provide a simple and user-friendly interface for interacting with the system.
- **External API Integration:**
  Fetches real-time sports data using TheSportsDB.
- **Containerized Deployment:**
  Uses Docker to package the application along with all dependencies, ensuring consistent behavior across environments.
- **Cloud Deployment:**
  Hosted on Amazon Web Services for remote accessibility and scalability.
- **Container Orchestration:**
  Utilizes Kubernetes to manage multiple instances, load balancing, and scaling.
- **Environment Configuration Management:**
  Secures API keys and configuration using environment variables and .env files.
- **Modular Architecture:**
  Ensures easy maintenance and scalability by separating AI logic, UI, and API integrations.

2.2 Overall Architecture / Workflow

The The system follows a layered architecture that integrates frontend interaction, AI processing, API communication, and cloud-based deployment.

Workflow Description:

1. The user interacts with the application through the Streamlit-based web interface.
2. The input query is processed by AI agents built using CrewAI and LangChain.
3. The agents use reasoning techniques (ReAct, tool calling) to determine the required action.
4. If needed, the system fetches real-time data from TheSportsDB API.
5. The processed result is generated using OpenAI models.
6. The response is sent back to the frontend and displayed to the user.
7. The entire application runs inside a Docker container.
8. The container is deployed on AWS EC2 and managed using Kubernetes for scaling and availability.



2.3 Tools & Technologies Used

The project utilizes a combination of programming languages, frameworks, APIs, and DevOps tools to build and deploy the system.

Programming Language

- Python – Core development language for backend and AI logic

AI Frameworks

- CrewAI – Used for implementing agent-based workflows
- LangChain – Used for chaining LLM calls and tool integration

LLM Integration

- OpenAI API – Used for generating intelligent responses

Frontend

- Streamlit – Used to build interactive UI

APIs

- TheSportsDB – Provides real-time sports data

Libraries

- requests – For API communication
- python-dotenv – For environment variable management

DevOps Tools

- Docker – For containerizing the application
- Kubernetes – For deployment and scaling
- Git – Version control system

Cloud Platform

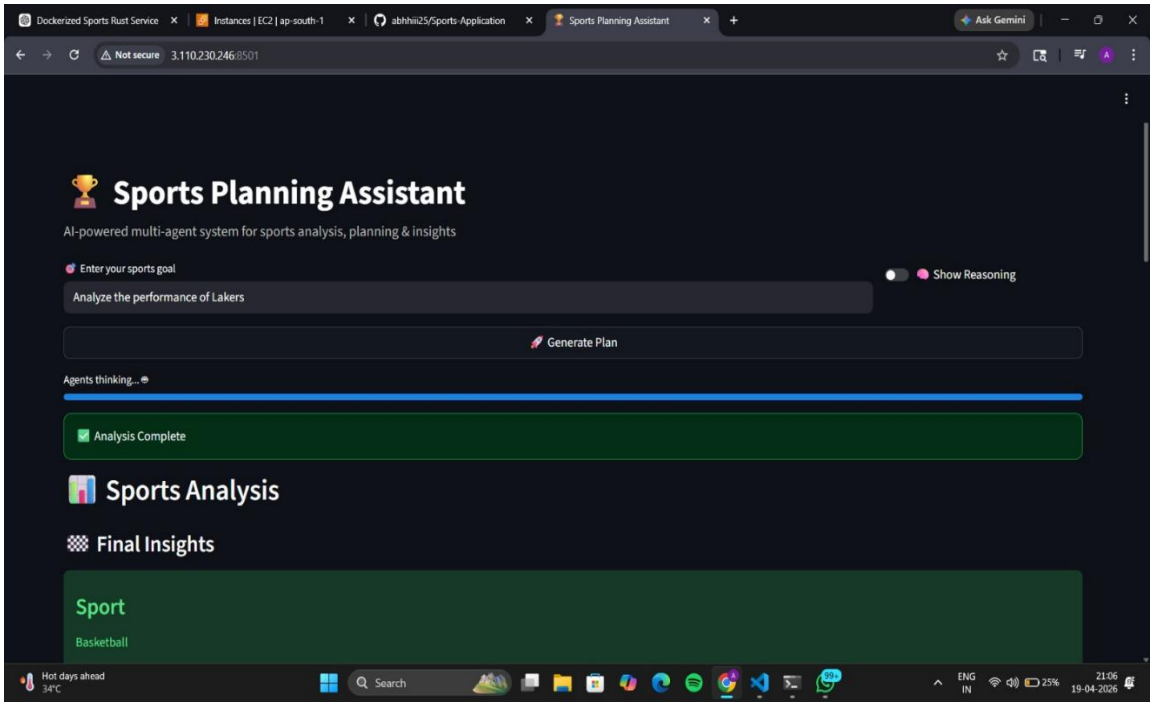
- Amazon Web Services – For hosting and deployment

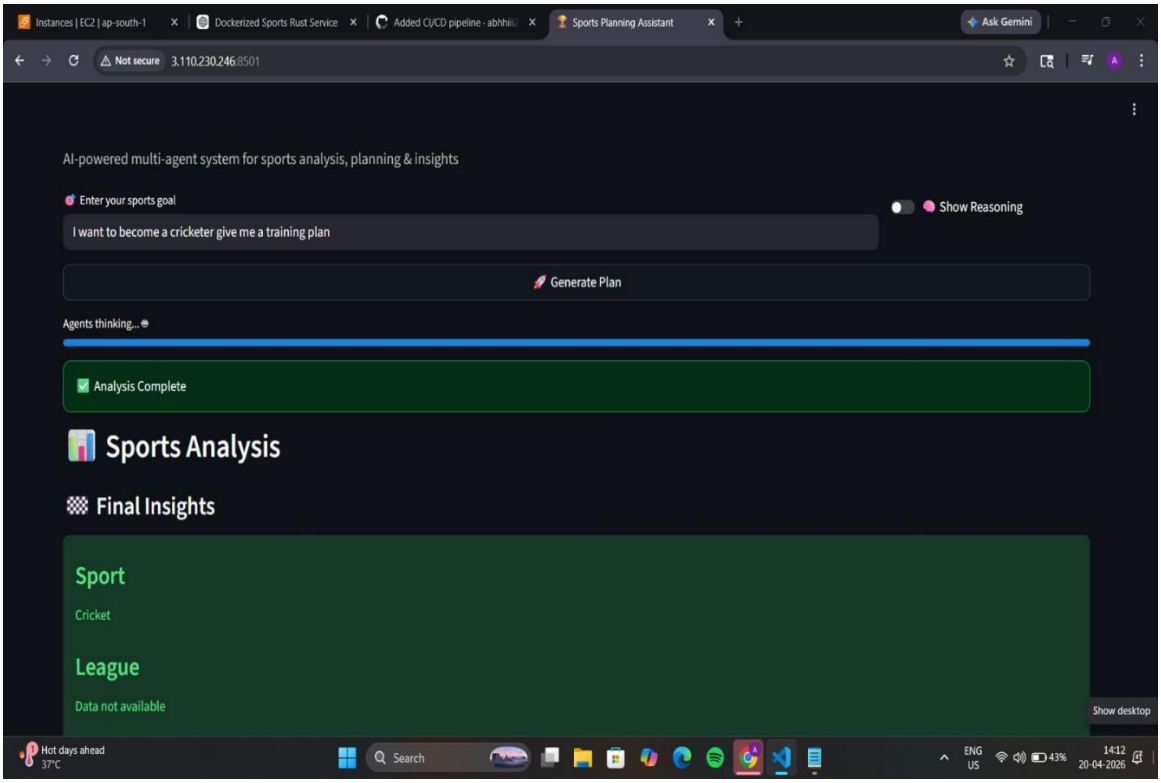
Development Tools

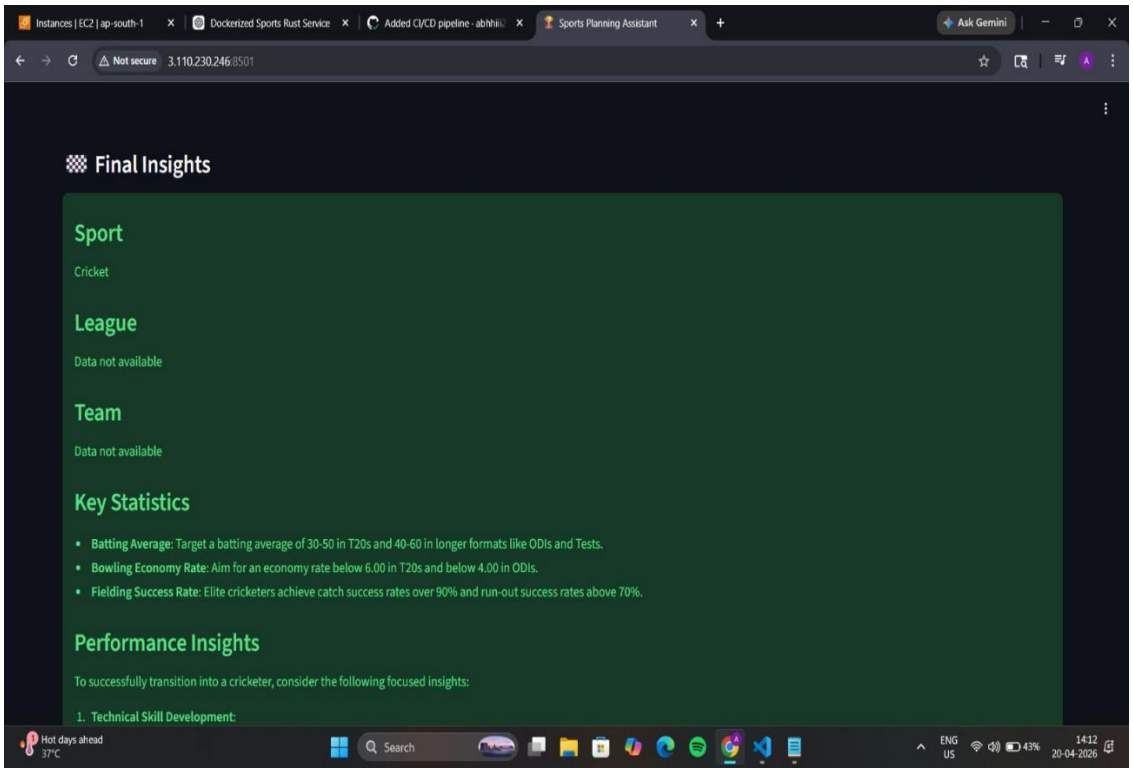
- Visual Studio Code – Development environment
- Virtual Environment – Dependency isolation

3.Result and Output

3.1Screenshots / outputs


Sports Planning Assistant
AI-powered multi-agent system for sports analysis, planning & insights
Enter your sports goal
Analyze the performance of Lakers
Show Reasoning
Generate Plan
Agents thinking...
Analysis Complete
Sports Analysis
Final Insights
Sport
Basketball


AI-powered multi-agent system for sports analysis, planning & insights
Enter your sports goal
I want to become a cricketer give me a training plan
Show Reasoning
Generate Plan
Agents thinking...
Analysis Complete
Sports Analysis
Final Insights
Sport
Cricket
League
Data not available



3.2. Reports / dashboards / models

The project includes an interactive dashboard and AI-based processing models that enable users to retrieve and analyze sports-related information efficiently.

Dashboards

The user interface is developed using Streamlit, which acts as a real-time interactive dashboard. It allows users to input queries and view dynamically generated responses.

Key dashboard features include:

- **User Query Input Panel:**
  Allows users to enter sports-related queries such as match details, team information, or analysis requests.
- **Dynamic Response Display:**
  Displays AI-generated outputs instantly based on user input.
- **Real-time Data Fetching:**
  Integrates with TheSportsDB to provide up-to-date sports data.

- **Simple and User-Friendly Interface:**

Ensures accessibility for users without technical expertise.

Reports

The system generates dynamic, on-demand reports instead of static predefined reports. These reports are created based on user queries and AI processing.

Types of reports include:

- Match summaries and analysis
- Team performance insights
- Player-related information (if available via API)
- General sports-related question answering

The reports are generated using AI models powered by OpenAI, ensuring accurate and context-aware responses.

Models

The core intelligence of the system is based on agent-driven AI models.

- **Agent-Based Model:**

Implemented using CrewAI, where multiple agents collaborate to process queries.

- **LLM Integration:**

Uses OpenAI models to generate natural language responses.

- **Workflow Orchestration:**

Managed using LangChain to connect tools, APIs, and reasoning steps.

- **ReAct Reasoning Approach:**

Enables the system to think, decide, and act (e.g., call APIs) before generating final responses.

- **Tool Calling Mechanism:**

Allows the AI to fetch external data (e.g., from TheSportsDB) when required.

These models work together to provide intelligent, context-aware outputs.

3.3 Key Outcomes

The project successfully demonstrates the integration of AI and DevOps practices to build a scalable and portable application.

Technical Outcomes

- Successfully developed an AI-powered sports service application
- Implemented agent-based AI using CrewAI and LangChain

- Integrated external sports data using TheSportsDB API
- Containerized the application using Docker
- Deployed the application on Amazon Web Services EC2
- Implemented container orchestration using Kubernetes

Performance Outcomes

- Achieved consistent application behavior across environments
- Reduced dependency conflicts through containerization
- Enabled scalability through Kubernetes deployment
- Improved response generation using AI-based processing

Learning Outcomes

- Gained practical knowledge of DevOps tools and practices
- Understood containerization and orchestration concepts
- Learned cloud deployment strategies using AWS
- Explored real-world implementation of agent-based AI systems

Practical Impact

- Demonstrates a real-world use case of combining AI with DevOps
- Provides a scalable solution for deploying AI applications
- Can be extended into a full production-level system with additional features

4. Conclusion

This project successfully demonstrates the integration of Artificial Intelligence with modern DevOps practices to build a scalable and portable application. The AI-powered sports service system, developed using frameworks such as CrewAI and LangChain, provides intelligent and dynamic responses to user queries.

The use of Docker ensured that the application and its dependencies were packaged into a consistent and portable environment, eliminating issues related to system configuration and compatibility. Deployment on Amazon Web Services enabled remote accessibility and reliable hosting, while Kubernetes facilitated efficient container management, scalability, and high availability.

The project also highlights the effectiveness of integrating external APIs such as TheSportsDB with AI models powered by OpenAI to deliver real-time, context-aware insights.

Overall, the system addresses key challenges in deploying AI applications by providing a structured, scalable, and efficient solution. It serves as a practical example of how DevOps methodologies can enhance the development, deployment, and maintenance of modern AI-driven systems.

5. Future Scope & Enhancements

While the current implementation demonstrates the core functionality and deployment pipeline, there are several opportunities to enhance and extend the project further.

Technical Enhancements

- **Advanced CI/CD Pipeline:**

Implement automated build, testing, and deployment using tools like GitHub Actions or Jenkins for continuous integration and delivery.

- **Improved Security Measures:**

Introduce Kubernetes secrets, role-based access control (RBAC), and secure API gateway integration to enhance system security.

- **Load Balancing & Auto Scaling:**

Configure advanced scaling policies and load balancing mechanisms within Kubernetes for handling high traffic.

- **Monitoring & Logging:**

Integrate monitoring tools such as Prometheus and Grafana for real-time performance tracking and logging.

Application Enhancements

- **Expanded Data Sources:**

Integrate additional sports APIs to provide more comprehensive and diverse data.

- **Personalized User Experience:**

Implement user authentication and personalized recommendations based on user preferences.

- **Enhanced UI/UX:**

Improve the frontend beyond Streamlit by using modern frameworks for a more interactive and responsive interface.

- **Multi-language Support:**

Enable the system to support multiple languages for a broader audience.

AI Enhancements

- **Fine-tuned Models:**

Use domain-specific fine-tuning of OpenAI models for improved accuracy in sports-related queries.

- **Advanced Agent Collaboration:**

Enhance agent workflows in CrewAI for more complex reasoning and decision-making tasks.

- **Predictive Analytics:**

Add features for match predictions, player performance forecasting, and trend analysis.

Deployment Enhancements

- **Production-grade Kubernetes Setup:**

Move from local or basic cluster setup to managed Kubernetes services (e.g., AWS EKS).

- **Custom Domain & HTTPS:**

Deploy with a custom domain and SSL certificates for secure access.

- **Global Deployment:**

Use multiple cloud regions for better availability and reduced latency.

.